



edunet
foundation

Deforestation Detection(Fire Classification)

PRESENTED BY

STUDENT NAME:- UDDIP BISHT

COLLEGE NAME:- Panipat Institute of Engineering & Technology

DEPARTMENT:- B.TECH (CSE)

EMAIL ID:- uddipbisht2004@gmail.com

AICTE Internship Student Registration ID:- STU668e4423c86ed1720599587

AICTE Internship ID:- INTERNSHIP_1748923002683e727a876ea

Learning Objectives

Learning Objectives:-

- Understand and analyze MODIS satellite fire data specific to India.
- Learn preprocessing techniques for handling real-world geospatial datasets.
- Apply feature scaling and encoding for machine learning readiness.
- Build and evaluate a classification model to predict fire types.
- Deploy a machine learning model using Streamlit for real-time predictions.
- Tackle challenges related to large file handling and model integration in web applications.



Tools and Technology used

Tools and Technology Used:-

- **Python** – Core language for data processing, modeling, and deployment.
- **Pandas & NumPy** – For data cleaning, manipulation, and numerical computations.
- **Scikit-learn** – To build and train machine learning classification models.
- **Joblib** – For saving and loading large model and scaler files efficiently.
- **Streamlit** – To create an interactive web application for fire type prediction.
- **Matplotlib & Seaborn** – For data visualization and exploratory data analysis.
- **Google Drive** – Used for sharing large model files exceeding GitHub's size limit.
- **Jupyter Notebook** – For exploratory analysis and model development.

Methodology

Methodology:-

1) Data Collection

- Acquired MODIS satellite fire data for India from the years 2021, 2022, and 2023 in CSV format.

2) Data Preprocessing

- Merged datasets and cleaned missing or invalid values.
- Converted categorical variables (e.g., confidence level) into numerical format.
- Scaled numerical features using standard scaling for uniformity.

3) Exploratory Data Analysis (EDA)

- Visualized data distributions and correlations to understand feature relevance.
- Identified patterns and anomalies using histograms, heatmaps, and pair plots.

4) Model Building

- Built a classification model using machine learning algorithms (e.g., Random Forest or XGBoost).
- Split data into training and testing sets for model evaluation.
- Optimized hyperparameters for best performance.

5) Model Evaluation

- Assessed the model using accuracy, precision, recall, and F1-score.
- Selected the best-performing model for deployment.

6) Model Deployment

- Integrated the trained model and scaler into a Streamlit app.
- Developed a user-friendly web interface for inputting fire data and predicting fire types.

7) Handling Deployment Constraints

- Managed model file size limitations by sharing large files via Google Drive.
- Ensured portability and usability across different platforms.

Problem Statement:

Problem Statement:-

India experiences numerous fire incidents annually, affecting forests, agriculture, and offshore areas. Accurately identifying the type of fire based on satellite data is crucial for timely response and effective disaster management.

However, raw satellite data such as from MODIS (Moderate Resolution Imaging Spectroradiometer) can be complex and requires processing and classification to determine the nature of the fire. Manual analysis is not feasible at scale due to the volume and complexity of data.

The objective of this project is to develop a machine learning-based system that can classify different types of fires in India (e.g., vegetation fire, offshore fire, and other land sources) using MODIS satellite data, and deploy it via a user-friendly web application for real-time prediction.

Solution:

Solution

To address the problem of identifying fire types from complex satellite data, the following solution was implemented:-

1. Data Integration and Cleaning

- Collected MODIS satellite fire data for India (2021–2023).
- Cleaned and merged datasets to form a unified dataset for modeling.

2. Feature Engineering

- Selected key features such as brightness, brightness_T31, FRP, scan, track, and confidence.
- Encoded categorical features and applied feature scaling for model compatibility.

3. Model Development

- Trained a supervised classification model (e.g., Random Forest) to categorize fire incidents into vegetation fires, offshore fires, and other land-based sources.
- Evaluated model performance and fine-tuned it for accuracy and robustness.

4. Deployment with Streamlit

- Built a web application using Streamlit to make predictions based on user inputs.
- Integrated the trained model and scaler to enable real-time fire type classification.

5. Scalable Access

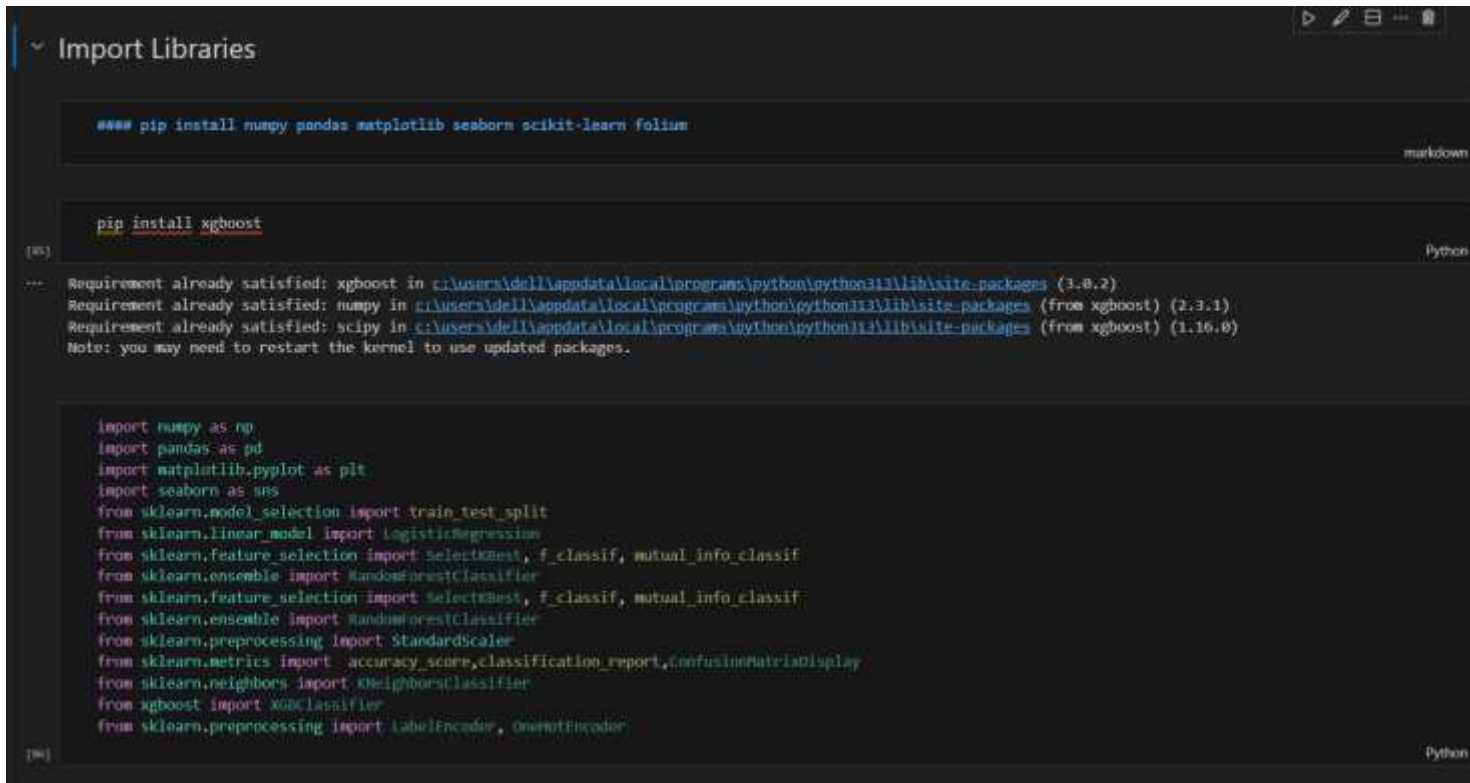
- Hosted large model files via Google Drive due to GitHub file size limitations.
- Designed the app to be lightweight, fast, and accessible from any device with internet access.

GITHUB REPO:-

https://github.com/uddipbisht/Classification_of_Fire_Types_in_India_Using_MODIS_Satellite_Data_week1.git

Screenshot of Output:

- IMPORT LIBRARIES



The screenshot shows a Jupyter Notebook interface with a dark theme. The notebook is titled "Import Libraries". It contains two code cells. The first cell is a markdown cell with the text "### pip install numpy pandas matplotlib seaborn scikit-learn folium". The second cell is a Python cell with the text "pip install xgboost". Below the code cells, the output of the second cell is displayed, showing the installation of xgboost and the update of numpy, pandas, and scipy. The output also includes a note to restart the kernel. Below the output, there is a third code cell containing a list of imports for various machine learning libraries.

```
### pip install numpy pandas matplotlib seaborn scikit-learn folium
```

pip install xgboost

Requirement already satisfied: xgboost in c:\users\dell\appdata\local\programs\python\python311\lib\site-packages (1.8.2)
Requirement already satisfied: numpy in c:\users\dell\appdata\local\programs\python\python311\lib\site-packages (from xgboost) (2.3.1)
Requirement already satisfied: scipy in c:\users\dell\appdata\local\programs\python\python311\lib\site-packages (from xgboost) (1.16.0)
Note: you may need to restart the kernel to use updated packages.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.feature_selection import SelectKBest, f_classif, mutual_info_classif
from sklearn.ensemble import RandomForestClassifier
from sklearn.feature_selection import SelectBest, f_classif, mutual_info_classif
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.neighbors import KNeighborsClassifier
from xgboost import XGBClassifier
from sklearn.preprocessing import LabelEncoder, OneHotEncoder
```

Screenshot of Output:

- Load the Dataset

```
Load the dataset

df1 = pd.read_csv('modis_2021_india.csv')
df2 = pd.read_csv('modis_2022_india.csv')
df3 = pd.read_csv('modis_2023_india.csv')

In [14]: ✓ 0s

df1.head() # print first 5 rows = df1.tail()

In [15]: ✓ 0s

***
  latitude  longitude  brightness  scan  track  acq_date  acq_time  satellite  instrument  confidence  version  bright_t31  frp  daylight  type
0  28.0993  96.9983    303.0      1.1  1.1  2021-01-01    409      Terra      MODIS         44      6.03    292.6  8.6  D  0
1  30.0420  79.6492    301.8      1.4  1.2  2021-01-01    547      Terra      MODIS         37      6.03    287.4  9.0  D  0
2  30.0879  78.8579    300.2      1.3  1.1  2021-01-01    547      Terra      MODIS         8       6.03    286.5  5.4  D  0
3  30.0408  80.0501    302.0      1.5  1.2  2021-01-01    547      Terra      MODIS         46      6.03    287.7  10.7 D  0
4  30.6565  78.9668    300.9      1.3  1.1  2021-01-01    547      Terra      MODIS         43      6.03    287.6  9.0  D  0

In [16]: ✓ 0s

df2.head()

In [17]: ✓ 0s

***
  latitude  longitude  brightness  scan  track  acq_date  acq_time  satellite  instrument  confidence  version  bright_t31  frp  daylight  type
0  30.1138  80.0756    300.0      1.2  1.1  2022-01-01    511      Terra      MODIS         7       6.03    288.4  7.1  D  0
1  23.7726  86.2078    306.1      1.6  1.2  2022-01-01    512      Terra      MODIS         62      6.03    293.5  10.4 D  2
2  22.2080  84.8627    304.8      1.4  1.2  2022-01-01    512      Terra      MODIS         42      6.03    293.3  5.8  D  2
3  23.7621  86.3946    306.9      1.6  1.2  2022-01-01    512      Terra      MODIS         38      6.03    295.2  9.3  D  2
4  23.6787  86.0891    303.6      1.5  1.2  2022-01-01    512      Terra      MODIS         52      6.03    293.1  7.2  D  2

In [18]: ✓ 0s

df3.head()

In [19]: ✓ 0s

***
  latitude  longitude  brightness  scan  track  acq_date  acq_time  satellite  instrument  confidence  version  bright_t31  frp  daylight  type
0   9.3280  77.6247    318.0      1.1  1.0  2023-01-01    821      Aqua      MODIS         62      6.03    305.0  7.6  D  0
1  10.4797  77.9378    313.8      1.0  1.0  2023-01-01    822      Aqua      MODIS         58      6.03    299.4  4.3  D  0
2  13.2478  77.2619    314.7      1.0  1.0  2023-01-01    822      Aqua      MODIS         55      6.03    302.4  4.9  D  0
3  12.2994  78.4085    314.3      1.0  1.0  2023-01-01    822      Aqua      MODIS         58      6.03    301.9  4.8  D  0
4  14.1723  75.5024    338.4      1.2  1.1  2023-01-01    823      Aqua      MODIS         88      6.03    305.3  41.5 D  0

In [20]: ✓ 0s

df = pd.concat([df1, df2, df3], ignore_index=True)
df.head()

In [21]: ✓ 0s

***
  latitude  longitude  brightness  scan  track  acq_date  acq_time  satellite  instrument  confidence  version  bright_t31  frp  daylight  type
0  28.0993  96.9983    303.0      1.1  1.1  2021-01-01    409      Terra      MODIS         44      6.03    292.6  8.6  D  0
1  30.0420  79.6492    301.8      1.4  1.2  2021-01-01    547      Terra      MODIS         37      6.03    287.4  9.0  D  0
2  30.0879  78.8579    300.2      1.3  1.1  2021-01-01    547      Terra      MODIS         8       6.03    286.5  5.4  D  0
3  30.0408  80.0501    302.0      1.5  1.2  2021-01-01    547      Terra      MODIS         46      6.03    287.7  10.7 D  0
4  30.6565  78.9668    300.9      1.3  1.1  2021-01-01    547      Terra      MODIS         43      6.03    287.6  9.0  D  0

In [22]: ✓ 0s

df.shape # rows and cols

In [23]: ✓ 0s

(271217, 15)

In [24]: ✓ 0s

df.info() # dt, mem

In [25]: ✓ 0s

***
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 271217 entries, 0 to 271216
Data columns (total 15 columns):
 #  Column  Non-Null Count  Dtype
---  ---
 0  latitude  271217 non-null  float64
 1  longitude  271217 non-null  float64
 2  brightness  271217 non-null  float64
 3  scan      271217 non-null  float64
 4  track     271217 non-null  float64
 5  acq_date  271217 non-null  object
 6  acq_time  271217 non-null  int64
 7  satellite  271217 non-null  object
 8  instrument  271217 non-null  object
 9  confidence  271217 non-null  int64
10  version   271217 non-null  float64
11  bright_t31  271217 non-null  float64
12  frp       271217 non-null  float64
13  daylight  271217 non-null  object
14  type      271217 non-null  int64
dtypes: float64(8), int64(3), object(4)
memory usage: 31.8+ MB
```


Screenshot of Output:

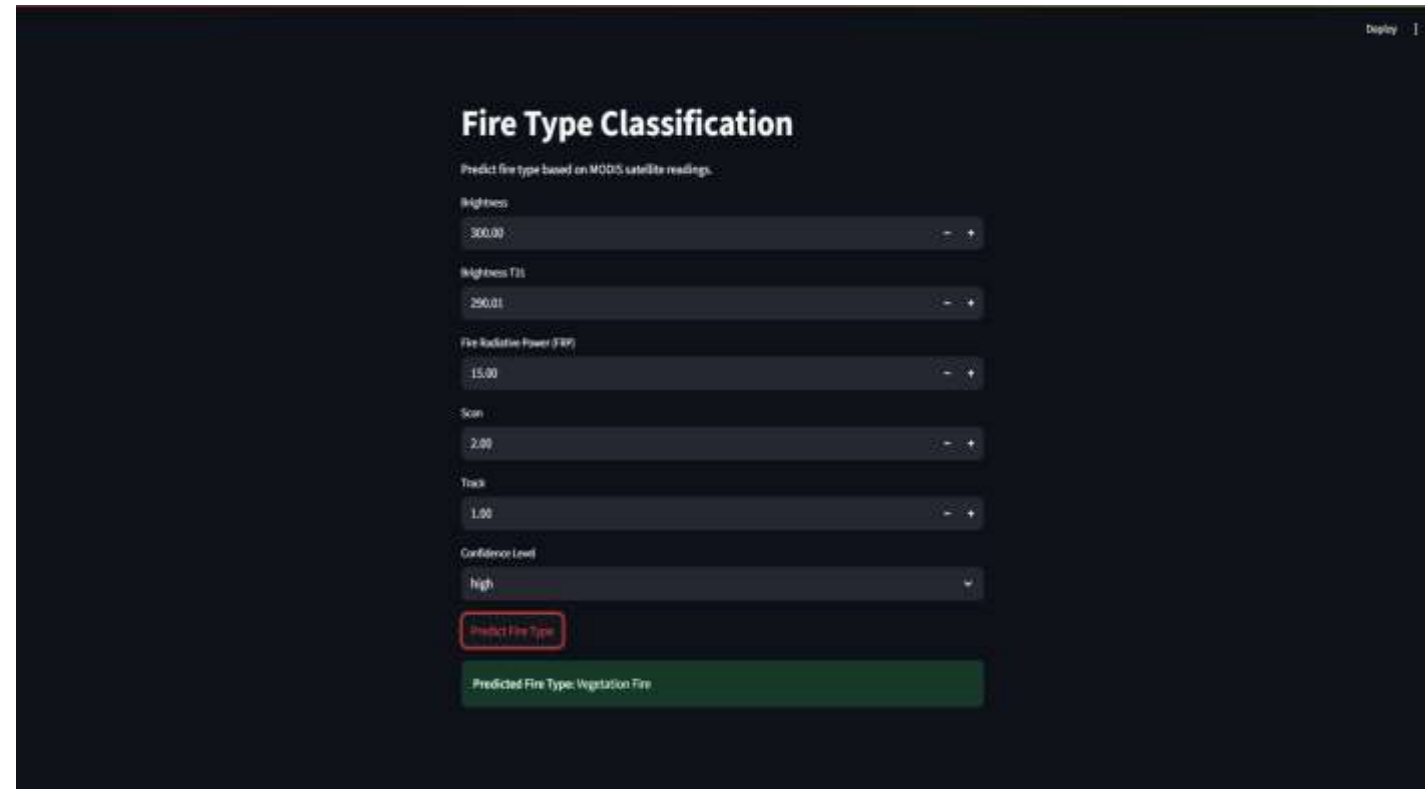
The screenshot from deployed **Streamlit app**, showing the following:

User inputs filled (Brightness, Brightness T31, FRP, Scan, Track, Confidence level)

A clicked “**Predict Fire Type**” button

The resulting output such as:

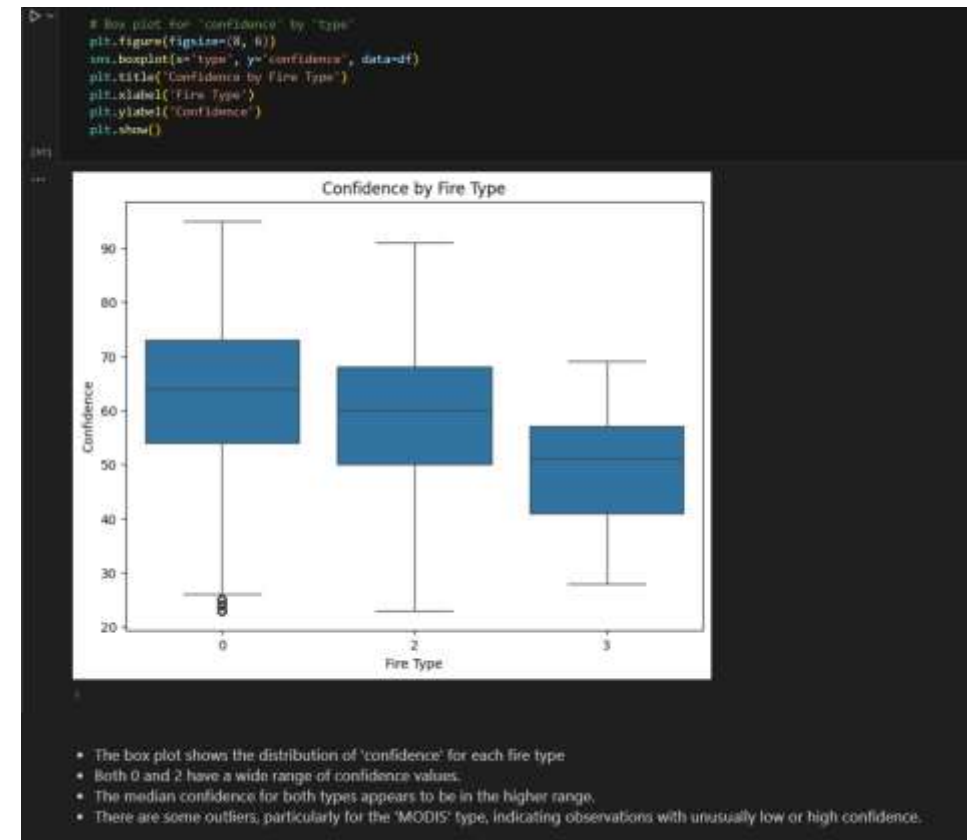
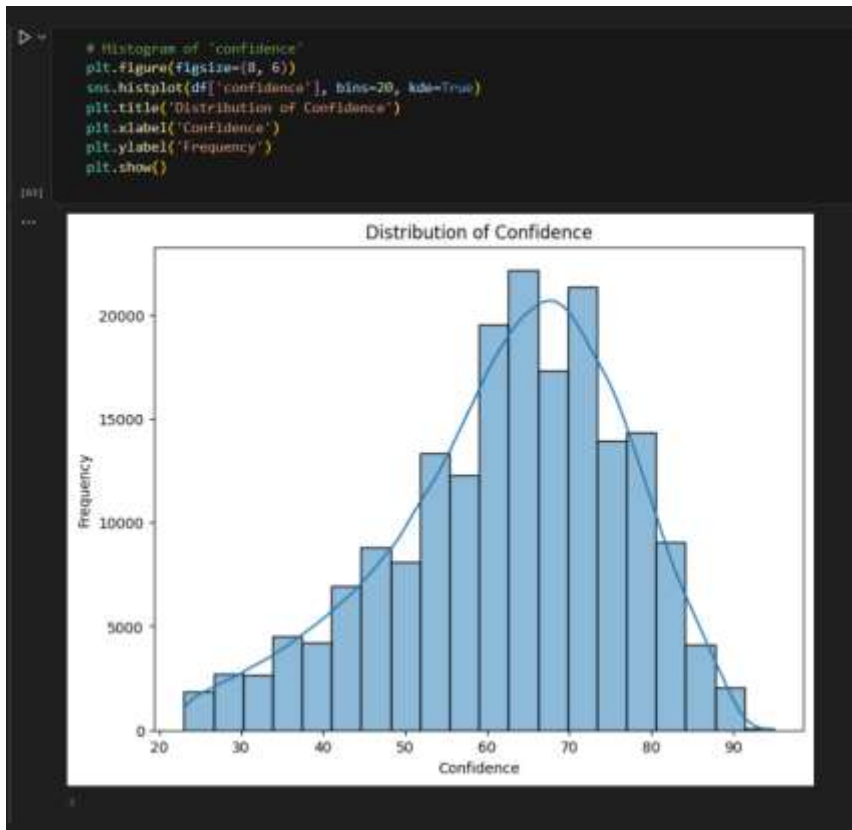
✓ **Predicted Fire Type: Vegetation Fire**



The screenshot displays a web application interface for fire type classification. The title is "Fire Type Classification". Below the title, a subtitle reads "Predict fire type based on MODIS satellite readings." The interface features six input fields, each with a label and a value: "Brightness" (300.00), "Brightness T31" (250.01), "Fire Radiative Power (FRP)" (15.00), "Scan" (2.00), "Track" (1.00), and "Confidence Level" (high). A red rectangular box highlights the "Predict Fire Type" button. Below the button, a green box displays the output: "Predicted Fire Type: Vegetation Fire".

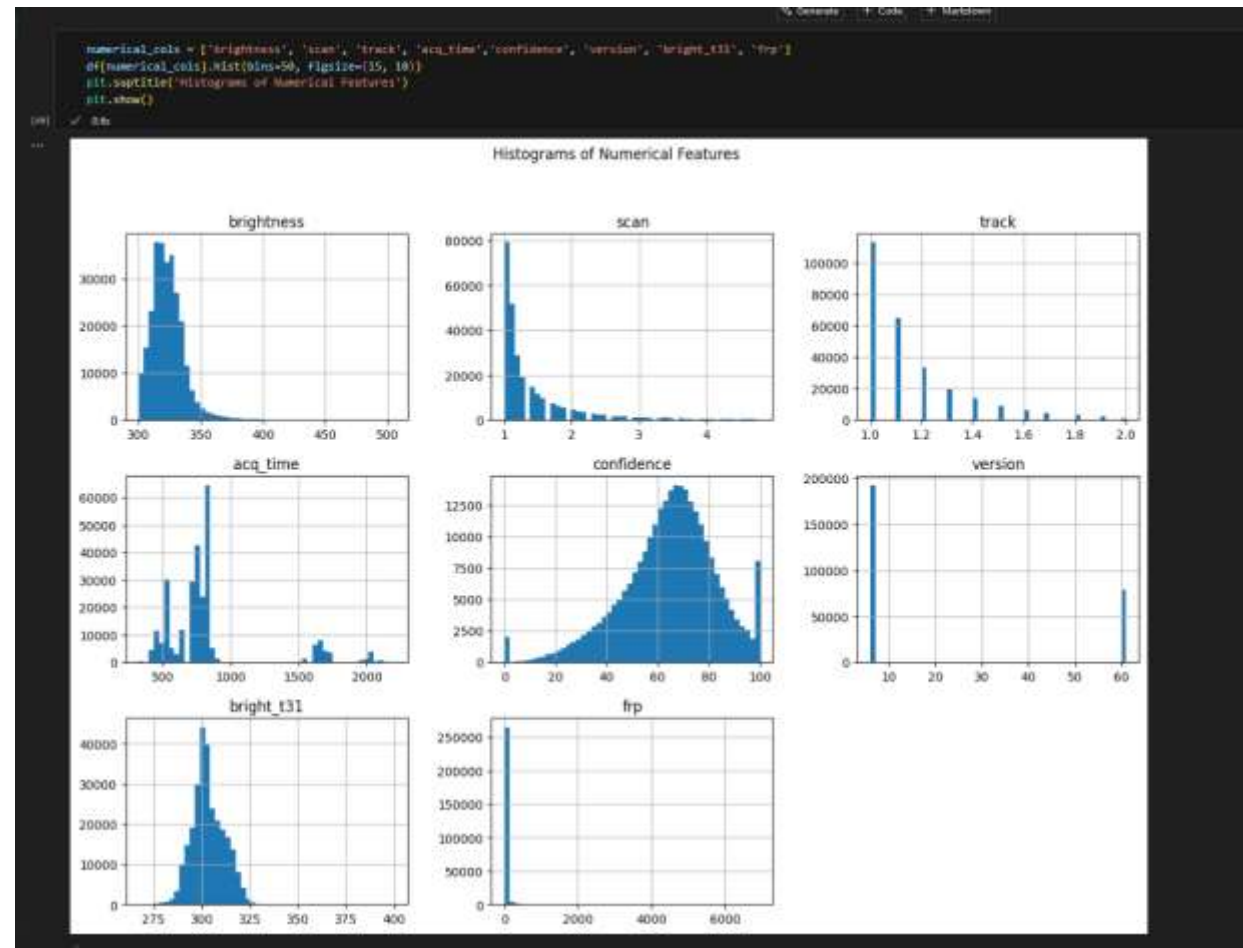
Screenshot of Output:

Exploratory Data Analysis (EDA):-



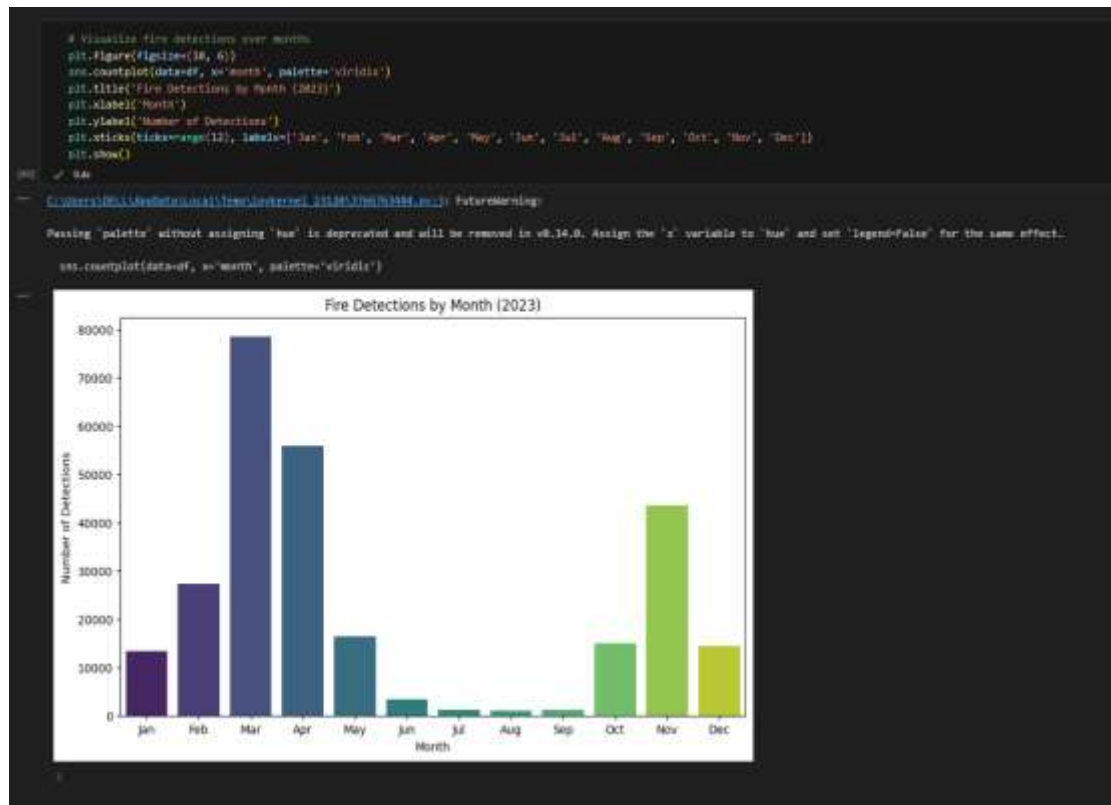
Screenshot of Output:

Histograms of Numerical Features

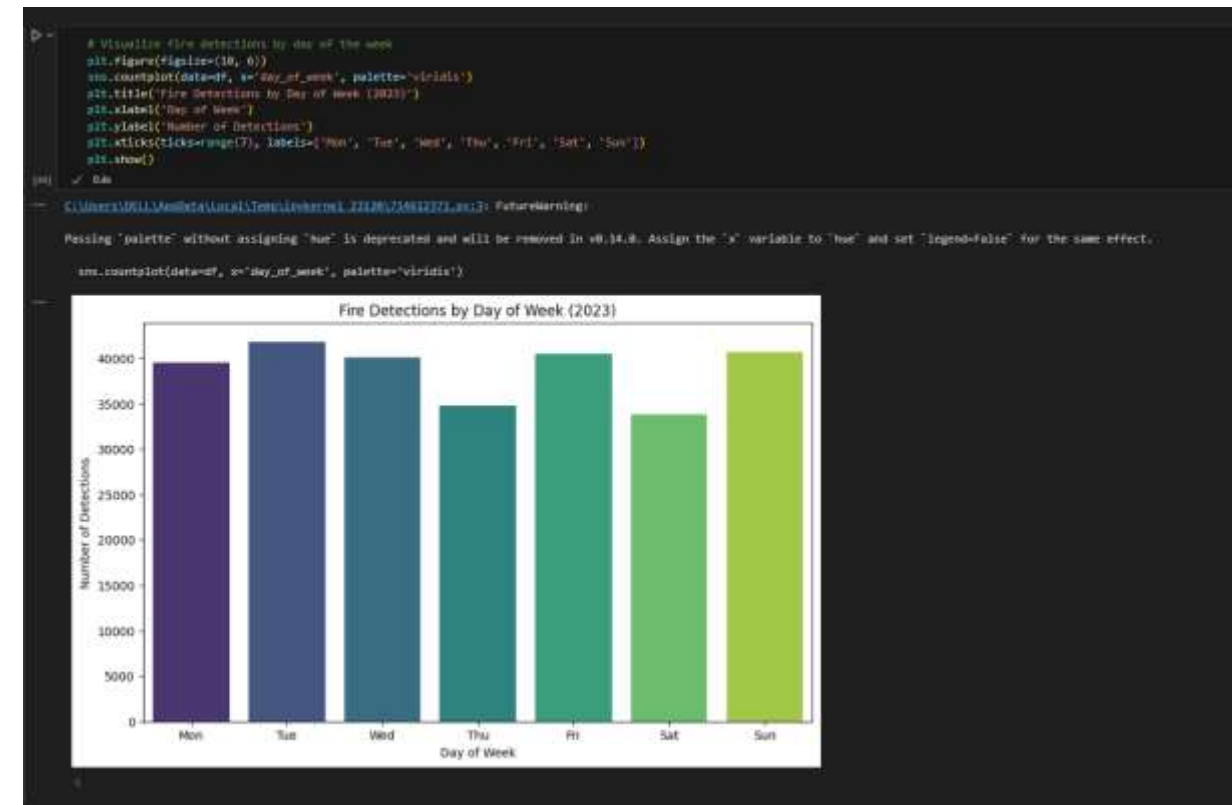


Screenshot of Output:

Visualize fire detections over months

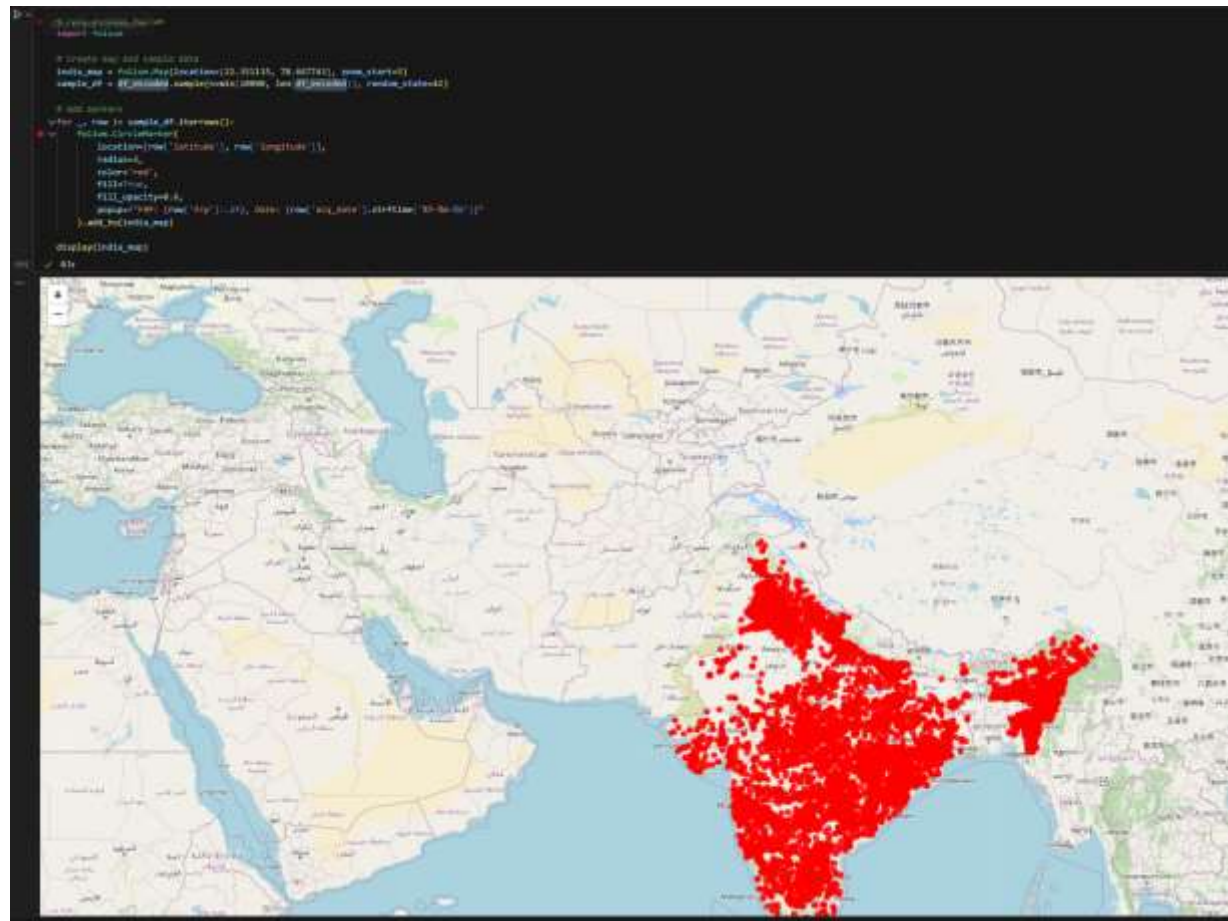


Visualize fire detections by day of the week



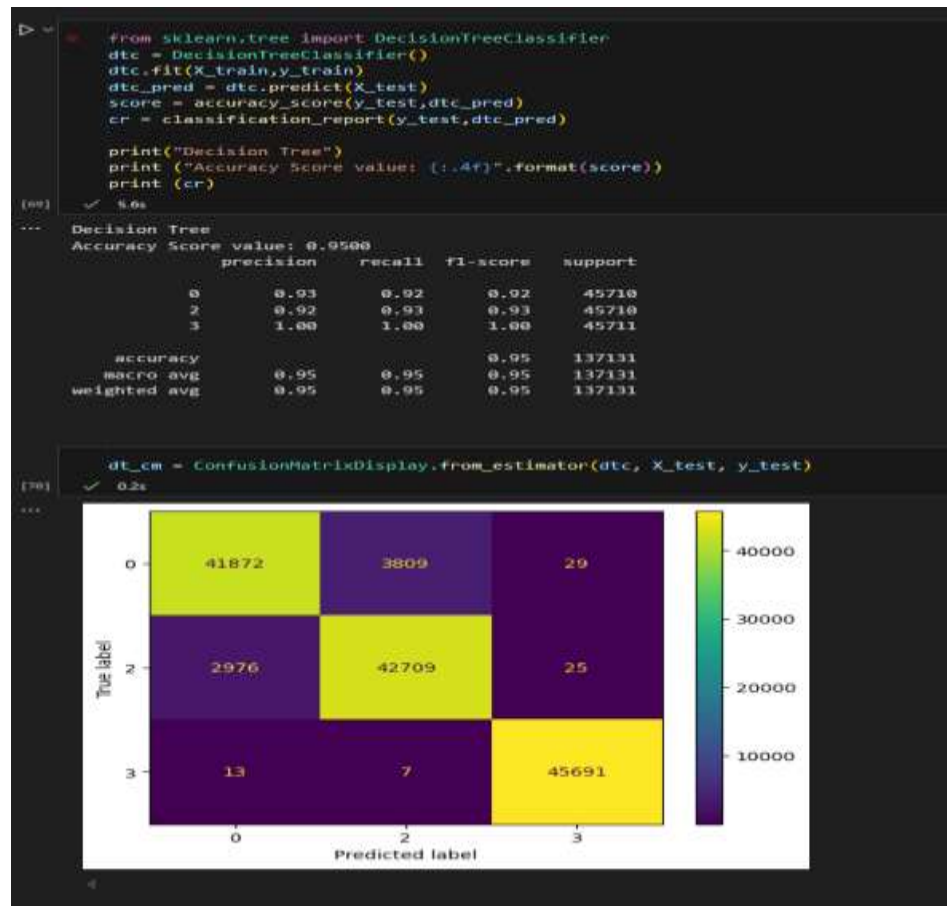
Screenshot of Output:

Folium map of India with red circle markers indicating fire locations based on the 'latitude' and 'longitude' columns of the dataset. Each marker represents a detected fire incident.



Screenshot of Output:

Decision Tree



Random Forest



Conclusion:

Conclusion:-

This project successfully demonstrates the application of machine learning for classifying fire types in India using MODIS satellite data. By leveraging key features such as brightness, fire radiative power (FRP), and confidence levels, we trained a reliable classification model capable of predicting fire categories with good accuracy.

The deployment of this model using Streamlit ensures ease of use and real-time predictions through an interactive web interface. Additionally, practical issues such as large model file size were effectively managed using external file hosting.

Overall, the project highlights the potential of combining remote sensing data with AI to support faster and more informed decision-making in fire monitoring and disaster response efforts.

Future Scope:-

- **Incorporate More Features:** Include additional environmental variables such as wind speed, humidity, and vegetation index to improve model accuracy.
- **Use Deep Learning Models:** Experiment with deep learning techniques (e.g., CNNs or LSTMs) for better pattern recognition in complex satellite data.
- **Real-Time Data Integration:** Connect the model with real-time MODIS or VIIRS satellite feeds for live fire detection and classification.
- **Mobile App Development:** Extend the application to mobile platforms for quick access by field officers and disaster response teams.
- **Alert System Integration:** Develop automated alert systems to notify concerned authorities when a specific fire type is detected.
- **Global Applicability:** Adapt and scale the model for fire detection in other countries or regions beyond India.